ChatGPT

Integrations modülü için çok adımlı entegrasyon akışları

Mevcut Integrations mimarisi ve kısıtlar

Elinizdeki Integrations modülü bugün “domain event → hook eşleşmesi → action yürütme → execution/DLQ ile izlenebilirlik” hattını merkezileştiriyor; Connector/Action/Hook/Execution/DLQ ana kavramları üzerine kurulu ve hem sync hem de RabbitMQ üzerinden async çalışmayı destekliyor. filecite turn0file0 Bu mimaride, asenkron akışta event’in `integration_events` kuyruğuna publish edilmesi ve worker’ın tüketip execution kaydını güncellemesi temel işletim modeli. filecite turn0file0

Mevcut “tek action” yaklaşımında esnekliği artıran kritik bileşen Payload Builder pipeline’ı: field mapping + static values + enrichment + defaults sırasıyla çalışarak harici isteğin payload’ını üretmeyi hedefliyor. filecite turn0file0 Ayrıca Action tarafında “response schema/success criteria” ile yanıtın başarı/başarısızlık değerlendirmesi runtime’da dinamik yapılabiliyor. filecite turn0file0 Admin UI tarafında connector/action/hook yönetimi; test-run; executions ve DLQ ekranları gibi uçtan uca gözlemlenebilirlik hedeflenmiş. filecite turn0file0

Bu genişletmenin en önemli kısıtı AYSTEK ekosistem standartları: Go backend’de DDD katmanları (model/dto/repository/service/handler/routes), seed zorunlulukları, menü girişleri ve özellikle UI metinlerinin 28 dilde translation seed ile eklenmesi gibi “otomatik uyulması gereken” kurallar var. filecite turn0file2 Integrations modülünün kendi içinde de (eventbridge, executor, worker vb.) belirlenmiş bir paket haritası ve RabbitMQ karar rehberi bulunuyor; Next.js tarafının AMQP’ye bağlanmaması gerektiği açık bir standart. filecite turn0file2

Son olarak, tasarım dokümanınız “mapping/DSL tarafında sınırsız scripting alanı vermenin (JS/Python gibi) kurumsal risklerini” vurguluyor; bunun yerine deklaratif ve bütçeli (time/output limit) expression + template yaklaşımı öneriyor (JMESPath + Mustache-benzeri). filecite turn0file1 Bu, çok adımlı akış tasarımında “adımlar arası veri taşıma” ihtiyacını çözerken, aynı zamanda güvenlik ve işletim risklerini düşük tutmak için kritik bir yön veriyor. filecite turn0file1

Hedeflenen genişletme ve kullanım senaryoları

İstediğiniz genişletme, bugün “event tetiklenince tek bir web servis çağır” modelinden, **birden fazla bağlantı/adım** ile sonuca giden **zincir (chain) tabanlı** bir modele evrilmek:

Token alma → token ile başka servis çağırıp ID alma → o ID ile veri çekme gibi ardışık bağımlı adımlar; ayrıca “if/case” ile koşullu dallanma (ör. response’a göre farklı endpoint’e gitme) gereksinimi var.

Gönderdiğiniz ekran görüntüsü, tam olarak bu ihtiyacın UI karşılığını gösteriyor: düğümler (nodes) arası bağlantılar (edges) ve “filter/choice” blokları üzerinden dallanma; ayrıca aksiyon bloklarının birden fazla farklı yoldan tetiklenebilmesi gibi bir grafik yapısı. (Bu örnek SendPulse benzeri bir flow builder yaklaşımı.)

Bu hedefi AYSTEK standartlarına uygun şekilde açarsak:

Bu genişletme **Integrations modülünün mevcut Connector/Action/Hook yatırımını bozmadan**, "Hook → Action" yerine "Hook → Chain (çok adımlı workflow)" bağını opsiyonel olarak eklemeli. Mevcut aksiyon tanımları, mapping ve response değerlendirme kabiliyetleri korunmalı; sadece "adımlar arası çıktıların sonraki adımın input'una girmesi" sağlanmalı.

Önerilen çözüm: Zincir tabanlı çalışma zamanı

Önerilen yaklaşımın özü: Integrations içine yeni bir **Chain** (veya Workflow) konfigürasyon objesi ekleyip, bunu **graf/tabanlı** bir "execution plan" olarak çalıştırmak.

Bu, endüstride "mesajı değişken bir adım dizisinden ardışık geçirmek" şeklinde bilinen Routing Slip yaklaşımına çok benzer: mesajın (bizde event + context) sıradaki endpoint/step'e taşınması ve her adımın çıktısının bir sonraki adıma etki etmesi. [1] Koşullu dallanma ihtiyacı ise Content-Based Router/ Choice deseninin doğrudan karşılığıdır: içerikteki değer/şarta göre farklı kola yönlendirme. [2]

Bu çözümle üç kritik hedefi aynı anda tutturursunuz:

Birincisi konfigürasyonla yönetim: bugün Connector/Action/Hook nasıl admin panelden yönetiliyorsa, Chain de aynı control-plane'de yönetilir. filecite turn0file0
İkincisi güvenli "expression + template" modeli: adımlar arası veri taşıma için **JMESPath** ve "logic-less template" yaklaşımıyla sınırlı ama güçlü bir DSL elde edersiniz (tasarım dokümanınızın önerisiyle uyumlu).
Üçüncüsü operasyonel izlenebilirlik: her adım execution kaydı üretebilir; chain run seviyesinde de özet/iz kaydı tutularak UI'da durum görselleştirilir (executions + DLQ yaklaşımınızı genişletir). filecite turn0file0

Bu noktada mimari karar: "Chain grafı" için ilk sürümde **DAG (acyclic)** kuralı (döngüsüz grafik) getirmenizi öneririm. Aksi halde loop/while destekleri; deadlock/sonsuz döngü; retry semantiklerinin karmaşıklaşması gibi işletim riskleri artar. Bu risk, RabbitMQ DLQ/TTL gibi mekaniklerle bile tamamen ortadan kalkmaz; ayrıca UI doğrulaması ve runtime guard gerekir. [3]

Backend tasarımı ve runtime detayları

Backend'de hedef: mevcut **Action executor** kabiliyetini yeniden kullanarak "chain runner" eklemek, worker/queue hattını bozmadan genişletmek.

Bu tasarımın ana bileşenleri:

Chain veri modeli
Yeni koleksiyon/objenin şunları taşıması yeterli olur:

- `tenant_id`, `site_id`, `env` scope (mevcut connector/action/hook scoping mantığıyla aynı eksen)
- `key` ve `name`
- `graph`: `nodes[]` ve `edges[]`
- `nodes[]`: her node `type` + `config` taşır
- `edges[]`: source → target ve (opsiyonel) `condition` / `case` bilgisi
- `status`: draft/active/disabled, versioning alanları (platform dokümanınız config plane vurgusunu destekler) filecite turn0file1

Node tipleri
MVP için minimum ama yeterli set:

- `start`: chain başlangıcı
- `action`: mevcut Action referansı + step-level mapping/override + output extraction
- `transform`: computed vars üretmek için expression çalıştırma (JMESPath)
- `branch`: if/else veya switch-case (choice)
- `end`: success/fail terminal

Burada pivot nokta: adımlar arası veri taşıma için ortak bir **Execution Context** tanımlamak. Önerilen context şeması:

- `event`: tetikleyici event payload + metadata (tenant/site/request_id/traceparent vb.)
- `vars`: zincir boyunca taşınan computed değerler
- `steps`: her node çalışınca `steps[node_id]` altına "seçilmiş çıktı" (response'tan JMESPath ile extract edilen alanlar) + özet status
- `secrets`: yalnızca server-side çözülür (UI/log'da mask)

Expression ve template standardı
Tasarım dokümanınızın yönlendirdiği gibi:

- **JMESPath**, JSON üzerinde query/extraction için formal spesifikasyonu, "structured data in/out" prensibi ve çoklu dil implementasyonları ile güçlü ve deterministik bir temel sağlar. 4
- **Mustache**, "logic-less templates" yaklaşımıyla template içine kontrolsüz iş mantığı gömülmesini azaltır. 5
- Bu ikilinin hibriti, "token'ı al ve header'a koy" gibi senaryolarda hem güvenli hem de yönetilebilir bir DSL sağlar (tasarım raporunuzun DSL v1 önerisiyle uyumlu). filecite turn0file1

İşletim modeli: hook → chain
Hook modelinde geriye uyumluluğu korumak için iki seçenek:

- Seçenek A: Hook içinde `target_type: action|chain` eklemek; action ise mevcut akış, chain ise yeni runner. Bu, mevcut hook'ları bozmadan ilerler. filecite turn0file0
- Seçenek B: Hook zaten "hangi action(lar) tetiklenecek" konseptine sahipse, bunu "action listesi" yerine "workflow" referansına genişletmek.

Seçenek A genelde daha az riskli ve migration maliyeti düşük.

Async çalışma ve retry/DLQ
Bugün async hat RabbitMQ worker'a dayanıyor. Chain runner'ı da aynı worker içinde çalıştırmak en uyumlu yaklaşım:

- Worker mesajı tüketir; hook/chain konfigürasyonunu yükler
- Chain runner node'ları sırayla yürütür
- Her `action` node'u mevcut HTTP executor'ı çağırır
- Hata olursa:
- "retryable" hatalarda belirlediğiniz retry/backoff profiline göre yeniden deneme
- limit aşılırsa DLQ'ya düşürme

RabbitMQ DLX (dead letter exchange) davranışı: mesajlar reject/nack, TTL expiration, queue limit aşımı gibi durumlarda dead-letter edilebilir; "neden" bilgisi (expired/rejected/maxlen vb.) header/annotation olarak taşınır. 3 Bu bilgiler chain DLQ triage ekranında çok kıymetli sinyal olur.

İdempotency ve tekrar çalışma semantiği
Chain'de en önemli "gerçek hayat" sorunu: retry edildiğinde 1. adım token aldı, 2. adım id aldı, 3. adımda hata oldu; tekrar çalışınca 2. adımın yan etkileri tekrarlar mı?

Burada iki kademe öneri:

- MVP: "chain run" düzeyinde idempotency key üretip aynı run'ın duplicate tüketimini engelleyin; buna ek olarak action node'ları için "side-effect" riskli endpoint'lerde vendor destekliyorsa idempotency header/body alanı koymayı konfigürasyonla yapın.
- v1: node bazlı idempotency (her step execution için ayrı key) + "başarılı adımı tekrar etmeme" (checkpointing). Bu, chain run'ı stateful yapar ve biraz daha büyük değişikliktir.

Endüstride Idempotent Consumer deseni "duplicate mesajları filtrelemek" için kullanılır; correlation key üzerinden tekrarları yakalar. [6] Tasarım dokümanınız da idempotency/dedup ihtiyacını özellikle async güvenilirlikte vurguluyor. filecite turn0file1

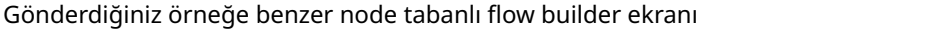
Observability ve kayıt stratejisi
Integrations modülü execution kayıtlarını zaten ana izleme kaynağı olarak kullanıyor. filecite turn0file0 Chain için iki seviyeli kayıt önerisi:

- ChainRun (üst kayıt): zincirin toplam durumu, hangi node'larda gezdi, toplam süre
- StepExecution (alt kayıtlar): her `action` node'u için request/response özetleri + status

Bunu ya ayrı koleksiyonla (chain_runs + step_executions) ya da mevcut executions koleksiyonunda `run_id` ve `node_id` alanlarıyla gerçekleştirebilirsiniz. Seçimi belirleyen pratik: bugün UI "Executions" ekranı action bazlıysa, run/step ayrımı UI'da daha iyi bir deneyim sağlar. filecite turn0file0

Next.js yönetim paneli ve görsel akış editörü

Aşağıdaki ekran, hedeflediğiniz UX'in iyi bir referansı: node'lar, bağlantılar ve filter/condition bloklarıyla görsel bir zincir editörü.



Gönderdiğiniz örneğe benzer node tabanlı flow builder ekranı

Bu tip editörler için React ekosisteminde en yaygın ve esnek yapılardan biri **React Flow**: node tabanlı editörler/diagramlar için özelleştirilebilir bir React bileşeni; custom node ve custom edge geliştirmeyi native olarak destekliyor. React Flow'un edge'leri SVG tabanlı çizmesi ve `nodeTypes/edgeTypes` yaklaşımı, sizin "branch edge'ine koşul etiketi koyma", "action node üzerinde test/preview butonları" gibi ihtiyaçlarınıza doğrudan olanak verir. [7]

UI tasarımını, mevcut Integrations ekran akışına uyacak şekilde şu şekilde konumlandırmak en düşük sürtünmeyle ilerler:

- Yeni sayfa: `/admin/integrations/chains` (liste) ve `/admin/integrations/chains/[id]` (editör)
- Hook ekranında: "Target" seçimi (Action mı Chain mi?) ve Chain seçilirse chain picker
- Chain editöründe:
- Sol palette node tipleri (Action/Transform/Branch/End)
- Canvas (React Flow)

- Sağ panel: seçilen node'un konfigürasyonu (action seçimi, template/expression alanları, output extraction)
- "Validate" ve "Test-run" (server-side simulator)

Mevcut platform dokümanınızın Action Test ve Hook Test kabiliyetleri zaten "preview + execute + response preview" yaklaşımını tanımlıyor; Chain test-run da aynı çizgide, sadece çok adımlı olacak. filecite turn0file0

Koşul node'ları için expression dili seçimi
Branch node koşullarını "JMESPath boolean expression" olarak tasarlamak pratik olur; çünkü JMESPath JSON üzerinde deklaratif extraction sağlar ve spesifikasyonu nettir. [8] Koşullu yönlendirme deseninin kavramsal karşılığı Camel Choice (content-based routing) olup, benzer bir "when/otherwise" mantığı UI'da da anlaşılırdır. [2]

Not: "if/case" alanını UI'da kullanıcı dostu yapmak için iki katman önerilir:

- Basit mod: operatör tabanlı (equals/contains/between) builder
- Gelişmiş mod: ham JMESPath (power users)

Bu ikili, kurumsal kullanıcı profili için iyi bir denge sağlar.

Codex ve Copilot için uygulanabilir iş planı

Aşağıdaki plan, Codex/Copilot'un repo'yu tarayıp "nerede neyi değiştirmeli" sorusuna net dosya hedefleri verecek şekilde hazırlanmıştır. Bu planın her adımı AYSTEK'in zorunlu standartlarına (DDD katmanları, seed, menü, 28 dil i18n) uymalıdır. filecite turn0file2

Tarama ve mevcut yapıyı çıkarma
Codex/Copilot'a şu aramaları yaptırın:

- Go: `internal/modules/integrations/` altındaki paketler (özellikle `eventbridge/`, `executor/`, `worker/`, `handler/`, `routes.go`) filecite turn0file2
- RabbitMQ: integrations event queue publish/consume noktaları; retry/DLQ mantığı
- Next.js: mevcut integrations admin sayfaları; Hook form (event key seçim UX'i); executions/dlq ekranları

Tasarım ve sözleşmeler
Backend ve frontend aynı "Chain schema" üstünde anlaşmalı. İlk sürüm için:

- Chain JSON şeması (nodes/edges) ve validasyon kuralları
- Tek `start` node
- En az bir `end` node
- DAG (cycle yok)
- Node sayısı üst limiti (örn. 50)
- Node tipleri ve config alanları (Action node için action_ref, template overrides, output extraction)

Backend geliştirme adımları
- Yeni Go model/dto/repository/service/handler/routes katmanlarını ekleyin (Integrations modülünün mevcut DDD desenine uygun). filecite turn0file2
- MongoDB koleksiyonları ve indeksler: chain listesi tenant/site scope'lu olmalı (tasarım dokümanındaki tenant/site izolasyon prensipleri).

- Admin API uçları:
- `GET/POST/PUT/DELETE /admin/integrations/chains`
- `POST /admin/integrations/chains/:id/validate`
- `POST /admin/integrations/chains/:id/test-run`
- Hook yürütme hattında `target_type` ekleyip (Action/Chain) chain runner'ı çağırma.

Runtime ve worker adımları
- Async modda worker mesajı tükettiğinde:
- Hook bul → target chain'i yükle → chain runner çalıştır
- Step bazlı execution kayıtlarını kaydet
- Retry/DLQ kararını node/chain politikasına göre ver
- DLQ ve retry mekaniklerini RabbitMQ DLX/TTL prensipleriyle uyumlu tutun; delayed retry gerekiyorsa TTL + DLX kombinasyonu kullanın. 3
- Duplicate tüketimi engellemek için idempotency yaklaşımını tasarlayın (en azından run seviyesinde); idempotent consumer fikriyle uyumlu correlation key üretin. 6

Next.js geliştirme adımları
- Menü ve sayfa ekleri: yeni sayfalar geldiğinde menü seed ve 28 dil translation key zorunluluğunu unutmayın. filecite turn0file2
- React Flow ile editör:
- Custom nodes: ActionNode, BranchNode, TransformNode, StartNode, EndNode 9
- Custom edges: "true/false" ve "case" etiketli edge'ler 10
- Node seçiminde sağ panel formu; form state → chain JSON
- Test-run paneli: server-side test-run çağırıp adım adım sonuçları görselleştirme (node üzerine status badge).
- Hook ekranı: Target selector (Action/Chain) ve chain seçimi.

Test ve kabul kriterleri
- Go unit test: chain validation (DAG, start/end), branch değerlendirme, context propagation. filecite turn0file2
- Entegrasyon testi:
- Mock HTTP server ile 3 adımlı senaryo (token → id → data)
- Branch senaryosu (response alanına göre farklı kola giden)
- Retry ve DLQ senaryosu (timeout/5xx)
- UI doğrulama: kaydet/yeniden yükle (chain JSON round-trip), validate hatalarını kullanıcıya gösterme, test-run sonuçlarının node bazında işaretlenmesi.

MVP kapsam önerisi
İlk iterasyonda kapsamı net sınırlamak, hız ve kaliteyi artırır:

- DAG + start/end + action + branch + transform
- Per-run idempotency (en azından duplicate consume engeli)
- Test-run (sync) ve gerçek çalıştırma (async)
- Execution ekranında "run_id" ile gruplayarak zincir izleme

v1'de checkpointing ve step-level retry eklenebilir.

Riskler, güvenlik ve geriye uyumluluk

Geriye uyumluluk
Mevcut hook'ların "tek action" davranışı bozulmamalı. Hook'a target_type eklemek ve default'u "action" yapmak bu riski minimize eder. filecite turn0file0

Güvenlik ve kötüye kullanım
Chain ile kullanıcıya "programlanabilirlik" verdiğiniz için, tasarım dokümanınızın uyardığı riskler büyür: kontrolsüz script alanı (RCE), CPU tüketimi (DoS), secret sızıntısı. filecite turn0file1 Bu yüzden:

- Yalnızca deklaratif expression/template: JMESPath + Mustache yaklaşımını koruyun. [11]
- Expression evaluation için süre/çıktı boyutu limitleri, derinlik limitleri (tasarım dokümanı "budget/sandbox" ihtiyacını vurguluyor). filecite turn0file1
- Secret'lar: UI ve log'da mask; runtime'da secure store/ref üzerinden çözümleme.
- SSRF kontrolleri: connector allowlist ve outbound güvenlik katmanları chain'de de aynen geçerli olmalı (chain "daha çok çağrı" demek, SSRF yüzeyinin büyümesi demek).

Operasyonel riskler
- Sonsuz döngü/loop: MVP'de DAG kuralı.
- Kuyruk birikmesi: worker ölçekleme; n8n'in queue mode modelinde ana instance trigger alır, worker'lar execution yapar; bu ölçekleme fikri sizin RabbitMQ worker havuzuyla uyumludur. [12]
- DLQ yönetimi: RabbitMQ DLX/TTL davranışlarını ve "neden" (expired/rejected vb.) sinyallerini UI triage'a taşıyın. [3]

Bu çerçeveyle, Integrations modülünüz "tek çağrı" entegrasyonlardan, yönetilebilir ve görsel olarak kurgulanabilir "çok adımlı zincir" entegrasyonlara evrilirken, mevcut mimari yatırımı (Action executor, mapping, response schema, execution/DLQ, RabbitMQ worker hattı) korunur ve AYSTEK standartlarına uyum sürdürülebilir.

---

[1] Routing Slip :: Apache Camel
https://camel.apache.org/components/4.8.x/eips/routingSlip-eip.html?utm_source=chatgpt.com

[2] Choice :: Apache Camel
https://camel.apache.org/components/4.10.x/eips/choice-eip.html?utm_source=chatgpt.com

[3] Dead Letter Exchanges | RabbitMQ
https://www.rabbitmq.com/docs/next/dlx?utm_source=chatgpt.com

[4] [8] [11] JMESPath Specification — JMESPath
https://jmespath.org/specification.html?utm_source=chatgpt.com

[5] {{ mustache }}
https://mustache.github.io/?utm_source=chatgpt.com

[6] Idempotent Consumer :: Apache Camel
https://camel.apache.org/components/4.8.x/eips/idempotentConsumer-eip.html?utm_source=chatgpt.com

[7] [10] Custom Edges - React Flow
https://reactflow.dev/learn/customization/custom-edges?utm_source=chatgpt.com

[9] Custom Nodes - React Flow
https://reactflow.dev/learn/customization/custom-nodes?utm_source=chatgpt.com

12 Configuring queue mode | n8n Docs
https://docs.n8n.io/hosting/scaling/queue-mode/?utm_source=chatgpt.com